

Biblioteca TWI/I2C para AVR – Driver com API completa

Nome: twi_i2c

Versão atual: 1.0 (implementação estável)

Objetivo: Fornecer uma interface limpa, assíncrona e robusta para comunicação TWI (I²C) em microcontroladores AVR (ATmega, ATtiny, etc.), com suporte completo a modos Master e Slave, sem polling bloqueante, apenas ISR.

1. Por que essa biblioteca existe?

A maioria das implementações TWI que vemos em projetos Arduino/AVR são:

- Bloqueantes (usam while até terminar)
- Polling (consomem CPU)
- Difíceis de depurar (sem callbacks, sem tratamento de erros claro)
- Não suportam Slave completo ou repeated start
- Misturam lógica de aplicação com driver

Essa biblioteca resolve, pois:

- Totalmente interrupt-driven (não bloqueia o processador)
- Callbacks para as situações (complete, error, receive, transmit)
- Suporte completo a Master Tx/Rx, Slave Tx/Rx e Repeated Start
- Timeout simples baseado em contador de interrupções
- Buffers internos configuráveis para Slave
- Debug condicional (zero overhead em produção)
- API limpa e consistente

É ideal para projetos que precisam:

- Responder como Slave (ex.: sensor custom, display inteligente)
- Comunicação com múltiplos dispositivos sem travar o código
- Rodar junto com outros periféricos (Timers, ADC, UART, etc.)

2. Estrutura da biblioteca

A biblioteca é dividida em 4 arquivos principais:

Arquivo	Função principal
twi.h	API pública – funções e tipos que o usuário usa
twi_types.h	Enums, structs e typedefs de callbacks
twi_config.h	Parâmetros configuráveis (clock, debug, buffers)
twi_internal.h	Estados internos, struct de transação, debug
twi.c	Implementação completa (ISR, funções, lógica)

O usuário final só precisa incluir twi.h e chamar as funções públicas.

3. Configuração básica (twi_config.h)

Antes de compilar, abra twi_config.h e ajuste conforme seu projeto:

```
#define TWI_DEBUG_MODE           0 // 1 = logs e variáveis de debug (mais flash/RAM)
#define TWI_DEFAULT_CLOCK       TWI_CLOCK_100KHZ
#define TWI_ENABLE_GENERAL_CALL 0 // Geralmente 0 (broadcast)
#define TWI_RX_BUFFER_SIZE      32 // Tamanho do buffer interno do Slave
```

4. Inicialização – o primeiro passo

```
#include "twi.h"
```

```
int main(void) {
    sei(); // Interrupções globais antes de twi_init

    // Exemplo Master
    twi_init(TWI_MODE_MASTER, 0, TWI_CLOCK_100KHZ);

    // Exemplo Slave (endereço 0x10)
    // twi_init(TWI_MODE_SLAVE, 0x10, TWI_CLOCK_100KHZ); // clock é ignorado no Slave

    // Registrar callbacks (se necessário)
    // twi_master_register_callbacks(meu_callback_complete, meu_callback_erro);
    // twi_slave_register_callbacks(meu_on_receive, meu_on_transmit);

    while(1) {
        // Seu código aqui – nunca bloqueie por muito tempo
    }
}
```

Observação importante: Sempre chame sei(); **antes** de inicializar o TWI, pois a ISR depende de interrupções globais.

5. Modo Master – como usar

5.1. Enviar dados (write):

```
uint8_t dados[] = {0x01, 0x02, 0x03};
twi_master_start_write(0x50, dados, 3); // endereço 0x50 (ex.: EEPROM)
// Não precisa esperar aqui – callback será chamado quando terminar
```

5.2. Ler dados (read)

```
uint8_t buffer[8];
twi_master_start_read(0x68, buffer, 8); // ler 8 bytes do endereço 0x68 (ex.: MPU6050)
```

5.3. Write + Read combinado (repeated start)

```
uint8_t reg_addr = 0x75;          // ex.: WHO_AM_I do MPU6050
uint8_t who_am_i[1];
twi_master_start_write_read(0x68, &reg_addr, 1, who_am_i, 1);
// Após o callback, who_am_i[0] deve ser 0x68 se o sensor estiver ok
```

5.4. Verificar se o barramento está livre

```
if (twi_get_bus_status() == TWI_BUS_IDLE) {
    // pode iniciar nova transação
} else {
    // ainda ocupado ou em erro
}
```

6. Modo Slave – como responder como escravo

6.1. Registrar callbacks

```
// Chamado quando alguém escreve para nós
void meu_on_receive(const uint8_t* dados, size_t len) {
    if (len >= 1) {
        if (dados[0] == 0x01) {
            // acende LED
        }
    }
}

// Chamado quando alguém quer ler de nós
size_t meu_on_transmit(uint8_t* buffer, size_t max_len) {
    if (max_len >= 4) {
        buffer[0] = 0xAB;
        buffer[1] = 0xCD;
        buffer[2] = 0xEF;
        buffer[3] = 0x01;
        return 4;
    }
    return 0;
}

int main(void) {
    twi_init(TWI_MODE_SLAVE, 0x10, TWI_CLOCK_100KHZ);
    twi_slave_register_callbacks(meu_on_receive, meu_on_transmit);
    // loop principal vazio
}
```

O Slave **não precisa** fazer nada no loop principal — tudo é tratado pela ISR.

7. Tratamento de erros e timeout

A biblioteca implementa **timeout automático** baseado em contador de interrupções (sem timers extras)

Após ~200 interrupções sem progresso (~100–200 ms em 100 kHz), envia STOP e seta TWI_ERROR_TIMEOUT

Erros comuns retornados nos callbacks:

TWI_ERROR_NACK → dispositivo não respondeu

TWI_ERROR_BUS_BUSY → outra transação ativa

TWI_ERROR_ARBIT_LOST → perdeu arbitragem (multi-master)

TWI_ERROR_TIMEOUT → travamento detectado

Exemplo de tratamento:

```
void meu_callback_complete(TWI_Status_t st, const uint8_t* data, size_t len) {
    if (st == TWI_OK) {
        // sucesso
    } else if (st == TWI_ERROR_TIMEOUT) {
        // Master travou ou não respondeu → pode retry
    } else {
        // outro erro
    }
}
```

8. Dicas de uso avançado

- **Evite chamar funções TWI dentro de ISRs** (pode causar reentrância)
- **Nunca bloqueie** por muito tempo no loop principal se usar Slave (o Master pode enviar dados a qualquer momento)
- **Teste com clock baixo primeiro** (100 kHz é mais tolerante a cabos longos)
- **Para multi-master** → a biblioteca já trata TW_MT_ARB_LOST
- **Economia de RAM** → diminua TWI_RX_BUFFER_SIZE se não precisar de buffers grandes no Slave
- **Debug** → ative TWI_DEBUG_MODE 1 para ver variáveis de monitoramento da ISR

9. Exemplos completo

Veja os arquivos na pasta “examples”.

10. Conclusão

Essa biblioteca foi projetada para ser:

- **Fácil de usar** (API intuitiva)
- **Robusta** (tratamento de erros, timeout, callbacks)
- **Eficiente** (sem polling, zero overhead em produção)
- **Flexível** (Master/Slave, debug opcional)

Se você seguir as boas práticas (sei() antes de twi_init(), não bloquear, tratar callbacks), ela vai funcionar de forma confiável em quase qualquer projeto AVR com I²C.

Boa programação e bons projetos!

Tiago Henrique dos Santos

Contatos:

Canal do YouTube: <https://www.youtube.com/channel/UCiIEwgsH0HEj3P2aA1-Sozw>

Instagram: <https://www.instagram.com/cienciaeletrica/>

E-mail: contato.cienciaeletrica@gmail.com